

Live Forex Volatility Tracker

Full-stack assessment across Milestones 0–2 — architecture, correctness, test rigour, accessibility, security posture and release readiness.

AUDIT DATE	25 August 2026
SCOPE	Entire repository at <code>C:\Applications (Projects)\Live Forex Volatility Tracker</code> — 48 Python modules, 22 TypeScript modules, 3 SQL migrations, 12 documents
METHOD	Static review plus live execution : every quantitative claim in this report was re-measured during the audit against Redis 7.0.15 and PostgreSQL 16.13, not taken from prior reporting
ENVIRONMENT	Linux container · Python 3.11.15 · Node 22.22.2 · 1 vCPU
STANDARD APPLIED	The project's own: <i>publish the number or delete the adjective</i>

Verdict

The engineering is strong. The project is not. The code under `packages/core` and `apps/` is the work of someone who understands distributed systems, numerical stability and accessibility at a level well above what a fellowship application normally shows. Nineteen findings are recorded below and none of them contradicts that.

Two of them, however, are disqualifying in their current state and neither is a code problem. **The repository is not under version control**, which means the CI pipeline it ships has never executed once and there is no commit history for a reviewer to read. **There is no licence**, which makes the work legally closed-source and therefore ineligible for the two open-source programmes it was built for.

Both are fixable in under an hour. Until they are, the quality of everything else is invisible to the audience it was written for.

1 Scope and method

This is an internal audit, written to be useful rather than flattering. It assumes the reader is the author and does not soften findings for an external audience.

1.1 What was examined

The complete repository as of the end of Milestone 2, Step 6: the dependency-free volatility and alerting core, the ingestor, the persistence worker, the FastAPI gateway, the React frontend, the SQL migrations, the container and CI definitions, and the documentation set including nine architecture decision records.

1.2 How evidence was gathered

Claims were verified by execution wherever execution was possible. Specifically, during this audit the following were run rather than assumed:

- The full Python suite against a live Redis and a live PostgreSQL 16 instance provisioned for the purpose, with coverage instrumentation extended to all four packages.
- The complete frontend suite, the type checker, the production build and the store benchmark.
- `mypy --strict` and both `ruff` gates.
- An `axe-core` pass against a faithful reconstruction of the assembled application shell — a structure the existing test suite never renders.
- Version-control status on both the working copy and the author's own machine.

1.3 Limits of this audit — stated, not buried

Three things could not be verified in the audit environment, and the findings below are qualified accordingly rather than presented as complete:

- **TimescaleDB was unavailable.** The package repository is blocked by the network policy in this environment. Coverage of the database path was therefore measured against plain PostgreSQL 16 with the hypertable, continuous-aggregate and compression DDL stripped. This measures the Python code correctly; it does *not* verify Timescale behaviour, and no claim here should be read as doing so.
- **No browser was driven.** Neither chart has been rendered in a real browser under test conditions by this audit or by the project's own suite. See finding M-1.
- **The live upstream feed was not exercised.** All measurements use the replay provider or synthetic feeds.

2 Summary of findings

2	4	8	5
CRITICAL	HIGH	MEDIUM	LOW

FINDINGS REGISTER — INDEX

ID	SEVERITY	FINDING	AREA
C-1	CRITICAL	Project is not under version control; CI has never executed	Process
C-2	CRITICAL	No licence — the work is legally closed-source	Legal / OSS

ID	SEVERITY	FINDING	AREA
H-1	HIGH	Coverage reporting excludes 65 % of the Python codebase	Testing
H-2	HIGH	Transport, REST and authentication layers have 0 % coverage	Testing
H-3	HIGH	Service-dependent tests skip silently; the suite still reports green	Testing / CI
H-4	HIGH	Confirmed accessibility violation in the shipped application shell	Accessibility
M-1	MEDIUM	No browser-level tests; both canvas render paths are never executed	Testing
M-2	MEDIUM	PerfMonitor is dead code — the main-thread claim is unmeasured	Frontend
M-3	MEDIUM	No React error boundary; one throw blanks the entire application	Frontend
M-4	MEDIUM	Load suite is excluded from CI; benchmark assertions never regression-checked	CI
M-5	MEDIUM	Frontend has no coverage measurement at all	Testing
M-6	MEDIUM	No contribution scaffolding — the thing GSoC organisations assess	OSS
M-7	MEDIUM	WebSocket authentication is off by default and unthrottled when on	Security
M-8	MEDIUM	No rate limiting on the compute-heavy estimator endpoint	Security
L-1	LOW	Supabase migration path has never been executed	Data
L-2	LOW	No HEALTHCHECK on the Python image	Ops
L-3	LOW	No skip link; keyboard users traverse the tablist on every load	Accessibility
L-4	LOW	No dependency vulnerability scanning — 286 transitive packages unaudited	Supply chain
L-5	LOW	Vestigial directories and stale delivery artefacts on disk	Hygiene

Read the distribution, not just the count. Both Critical findings and four of the eight Medium ones are *process and packaging* problems, not defects in the running system. Nothing in this audit found an incorrect volatility calculation, a broken state machine, a race in the ingest path or a lost message. The engineering holds up under examination; the scaffolding around it does not yet exist.

3 What is actually here

Measured line counts, excluding virtual environments, `node_modules`, build output and caches.

CODEBASE INVENTORY

AREA	FILES	LINES	CONTENTS
Python source	48	6,091	Core maths and alerting, ingestor, worker, API gateway
Python tests	18	3,160	Unit, integration, chaos, load
TypeScript source	22	2,312	Store, hooks, four visualisations, theming
TypeScript tests	13	1,626	Store, hooks, a11y, contrast, components
SQL migrations	3	475	Hypertables, aggregates, alert events
Documentation	12	775	9 ADRs, benchmarks, milestone plan

Source totals **8,403 lines** against **4,786 lines of test** — a test-to-source ratio of **0.57**. That ratio is healthy, and it is worth noting that it is honest: the tests are not padded with assertions on trivia. It is also, per finding H-2, distributed very unevenly.

TEST SUITE COMPOSITION — COUNTS VERIFIED BY COLLECTION

SUITE	TESTS	RUNS IN CI	REQUIRES
Backend — unit	109	Yes	Nothing
Backend — integration	34	Yes	Real Redis, real PostgreSQL
Backend — chaos	19	Yes	Real Redis, fault server
Backend — load	4	No — opt-in only	Real Redis
Frontend — Vitest	115	Yes	jsdom
Total	281	277 in CI	—

4 What holds up under examination

An audit that lists only problems is not an audit, it is a complaint. These are the things that survived deliberate attempts to break them, and they are the reason the findings register is about packaging rather than correctness.

4.1 The tests have caught real defects, repeatedly

This is the strongest evidence in the project, because it is evidence about the *process* rather than the artefact. Across Milestones 1 and 2 the suite caught six genuine defects that static review would not have:

DEFECTS FOUND BY TESTING, NOT BY READING

DEFECT	WHY IT MATTERED
Baseline thaw was a no-op	The detector thawed the baseline and re-froze it on the next line, so persistent stress could never be accepted as a new normal. Silent — the feature simply did nothing.

DEFECT	WHY IT MATTERED
Seasonality reference drifted around the clock	The global level was an EWMA over <i>arrival order</i> , so it measured time-of-measurement rather than time-of-day. Test oracle was also wrong (arithmetic vs. geometric mean); both were corrected.
Cold standby goes silent mid-crisis	The chaos test's control arm <i>failed to reproduce the assumed bug</i> . Probing found the real failure mode was worse: a promoted standby re-baselines on elevated data, concludes it is normal, and never emits the clear — leaving the database stuck on a stressed row. Six code comments telling the assumed story were corrected.
Light-palette contrast failure	Numeric WCAG assertion caught <code>up</code> and <code>down</code> sitting 1.10:1 apart in luminance — one colour to a deuteranope, in the one place (candle direction) where canvas offers no shape to fall back on.
Dangling <code>aria-controls</code>	axe caught a reference to an element the fixture never rendered.
<code>role="alert"</code> on <code><article></code>	ARIA forbids overriding an implicit role; the toast used the wrong element.

4.2 Test quality has been verified, not assumed

Eight load-bearing guarantees were mutation-tested — deliberately broken to confirm the suite fails. All eight were caught. More usefully, the exercise *found a weak test*: the z-pane's "band holds still" assertion originally passed for the wrong reason, and was retightened to a value that actually discriminates. A suite that has been attacked by its own author is worth considerably more than one that has only ever been run in the happy direction.

4.3 Real services rather than mocks

Integration and chaos tests run against genuine Redis and PostgreSQL. The behaviours under test — consumer-group pending lists, `XAUTOCLAIM` idle windows, `MINID` trimming semantics, lease expiry — are precisely the things a mock would get wrong in the same direction as the assumption being tested. This decision is correct and it has already paid: the `MINID ~` test initially failed because Redis trims whole macro-nodes, a fact no mock would have surfaced.

4.4 Quantified claims

`docs/benchmarks.md` carries measured numbers with a stated environment, and — rarer and more creditable — an explicit "*what is not benchmarked yet*" section that names the end-to-end latency figure as unmeasured rather than estimating it. The replay-provider section reports an 840/s achieved figure against a 2,000/s target and explains the shortfall as `asyncio.sleep()` granularity instead of quietly reporting the target.

4.5 Type and lint discipline

`mypy --strict` passes across all 48 Python modules with no ignores outside a single annotated third-party gap. `ruff` passes both check and format across 79 files. `tsc --noEmit` passes under `strict` with `noUnusedLocals` and `noUnusedParameters`. All four were re-run during this audit.

5 Findings register

Critical

C-1 The project is not under version control

CRITICAL

EVIDENCE

```
$ git rev-parse --is-inside-work-tree    # working copy
NOT A GIT REPOSITORY
$ git rev-parse --is-inside-work-tree    # author's machine
NOT A GIT REPOSITORY
```

IMPACT

Three consequences, in descending order of severity.

- **The CI pipeline has never run.** GitHub Actions executes only against a GitHub repository. `.github/workflows/ci.yml` defines three jobs with service containers, schema application and a compose smoke test — and not one of them has ever executed. Every statement in this project's history that the CI configuration is correct, including the recent verification that it parses, establishes only that the YAML is well-formed.
- **There is no commit history.** MLH Fellowship and GSoC reviewers read commit history directly; it is how they distinguish sustained work from a code dump. A repository whose entire history is one initial commit presents a 8,400-line codebase with no evidence of how it was built, which is the weakest possible framing for work that was in fact built incrementally across three well-defined milestones.
- **There is no backup and no rollback.** The sole copy is one directory on one machine. The `.gitignore` file is inert.

REMEDIATION

`git init`, then reconstruct history as a small number of honest, well-described commits along the milestone boundaries that already exist — scaffold, volatility core, alerting and hysteresis, persistence, frontend state architecture, visualisations. Push to GitHub and confirm the CI run goes green before anything else in this register is addressed, because until it does, several other findings cannot be verified at all.

C-2 No licence — the work is legally closed-source

CRITICAL

EVIDENCE

```
LICENSE                MISSING
LICENSE.md             MISSING
pyproject.toml         no license field
apps/web/package.json  no license field
```

IMPACT

Under the Berne Convention, a work with no licence is **all rights reserved** by default. Nobody may legally fork it, modify it, or contribute to it. This project was built specifically for two programmes whose entire purpose is funding open-source contribution; presenting them with a repository they are not permitted to use is a category error, and it is the kind of detail a reviewer notices immediately because it is the first file they look for.

REMEDIATION

Add `LICENSE` at the repository root. MIT or Apache-2.0 are the conventional choices; Apache-2.0 additionally grants an explicit patent licence, which some organisations prefer. Mirror the choice into the `license` field of both `pyproject.toml` and `apps/web/package.json`. Fifteen minutes of work, and it changes the project's status from unusable to contributable.

High

H-1 Coverage reporting excludes 65 % of the Python codebase

HIGH

EVIDENCE

```
pyproject.toml
[tool.coverage.run]
source = ["packages/core/fx_core"]

As configured   : 88 % of    859 statements
All 4 packages  : 64 % of 2,478 statements (882 missed)
```

IMPACT

The 88 % figure is not wrong, but it describes only the dependency-free maths core — the part of the system that was always going to be easy to cover, because it is pure functions over numbers. The measurement excludes the ingestor, the persistence worker and the entire API gateway: 1,619 of 2,478 statements, or 65 % of the Python code. A coverage number scoped to the easiest third of a codebase is a number that improves as the untested part grows, which is the opposite of what the metric is for.

REMEDIATION

Extend `source` to all four packages and accept the honest 64 %. Then add `fail_under` at, say, 60 % so it becomes a ratchet rather than a statistic. Publishing 64 % across the whole system is a stronger claim than 88 % across a curated subset, and a reviewer who checks will find the second framing far more convincing than the first.

H-2 Transport, REST and authentication layers have 0 % coverage

HIGH

EVIDENCE

Modules never executed by any test (528 statements, 21 % of the codebase):

fx_api/routers/stream.py	81	the WebSocket endpoint itself
fx_api/routers/health.py	42	/readyz – which the compose job asserts on
fx_api/routers/market.py	41	the entire REST surface
fx_api/ws/protocol.py	29	message validation
fx_api/ws/tickets.py	17	the authentication path
fx_api/main.py / deps / cfg	88	app construction and wiring
fx_ingestor/main.py	129	entrypoint and leader loop wiring
fx_worker/main.py	50	entrypoint and task supervision
fx_*/config.py	49	settings parsing

IMPACT

The conflating queue is tested to 98 % and the pipeline to 98 %, but the *endpoint that uses them* has never been invoked. Nothing verifies that a subscribe operation returns a snapshot, that the symbol cap is enforced, that a malformed frame produces a protocol-error close, or that an invalid ticket is rejected. The readiness probe that the compose smoke test depends on is itself untested, which means a regression there would be caught only by a job that has never run (see C-1). This is the classic shape of a well-tested system with an untested surface: every unit is correct and nothing proves they are wired together.

REMEDIATION

FastAPI's `TestClient` supports WebSockets directly, so this is cheaper than it looks. A focused file of roughly a dozen tests — hello frame, subscribe/snapshot, symbol-cap close, malformed-frame close, ticket accepted, ticket rejected, ping/pong, `/readyz` healthy and `/readyz` with a stale feed — would move this from 0 % to the majority of the branches that matter and close the single largest gap in the project.

H-3 Service-dependent tests skip silently; the suite still reports green

HIGH

EVIDENCE

```
tests/conftest.py
    pytest.skip(f"Redis not reachable at {REDIS_URL}: {exc}")
```

Observed during this audit – Redis stopped mid-session:
54 tests skipped, 0 failed, exit code 0

IMPACT

This was not a hypothetical; it happened during the audit and produced a passing run that silently omitted every integration and chaos test. The `conftest` docstring states the intent is to "skip loudly instead of passing vacuously", but the implementation is not loud: `pytest -q` renders skips as bare `s` characters and the exit code is zero. In CI, a service container that fails its health check, a changed port, or a wrong environment variable would reduce the suite to unit tests only and the badge would stay green. Given that the CI pipeline has never run at all (C-1), the first real execution is precisely when this failure mode is most likely.

REMEDIATION

Add a CI-only guard: with `FX_REQUIRE_SERVICES=1` set in the workflow, the fixtures should `pytest.fail` rather than skip. Locally the skip behaviour stays, so a contributor without Docker can still run the unit tier. One conditional in `conftest.py`, and it converts a silent degradation into a build failure.

H-4 Confirmed accessibility violation in the shipped application shell

HIGH

EVIDENCE

```
App.tsx
<main id="symbol-panel" role="tabpanel" aria-labelledby={`tab-${active}`}>

axe-core, run against this structure during the audit:
VIOLATIONS: aria-allowed-role (minor impact, 1 node)
"ARIA role tabpanel is not allowed for given element"

Corrected structure, re-run:
VIOLATIONS: NONE
```

IMPACT

`<main>` has an implicit `main` role that ARIA does not permit overriding. The delivered page therefore has **no main landmark at all**, so screen-reader users lose the primary navigation shortcut for skipping to content — which compounds L-3, the absent skip link.

The more important part of this finding is *why it shipped*. The project has a genuinely good accessibility suite: 11 axe assertions, numeric WCAG contrast verification, shape-plus-colour encoding, roving tabindex, timing controls that satisfy 2.2.1. Every one of those tests renders a component in isolation. None renders the assembled application, so a defect that exists only in the composition was structurally invisible to it. The suite is testing the pieces and asserting about the product.

REMEDIATION

Keep `<main>` as a plain landmark and move the tab panel to an inner `<div id="symbol-panel" role="tabpanel" tabIndex={0}>`. The `tabIndex` is required, not cosmetic: the panel contains no focusable elements, and the ARIA Authoring Practices Guide requires such a panel be focusable or keyboard users cannot reach its content. Verified axe-clean during this audit. Then add one shell-level a11y test so the class of defect cannot recur — that test is worth more than the fix.

Medium

M-1 No browser-level tests; both canvas render paths are never executed

MEDIUM

IMPACT

jsdom has no canvas implementation. `PriceChart` — Lightweight Charts, series markers, the full-height shading histogram, the live bar update path — is executed zero times by the suite. `ZScorePane`'s renderer runs only against a recording stub written for the purpose, which verifies the segmentation logic but proves nothing about pixels. Chart theming, the resize path, and the marker ordering requirement are all unverified. Playwright with Chromium is already available in this toolchain.

REMEDIATION

A small Playwright suite driving the app against the replay provider: assert the chart canvas has non-blank pixels, that a transition renders a marker, that the theme toggle repaints the series, and capture screenshots for visual regression. Roughly half a day, and it is the missing half of the frontend testing story.

M-2 PerfMonitor is dead code; the main-thread claim is unmeasured

MEDIUM

EVIDENCE

```
$ grep -rn "import.*perf" apps/web/src --exclude-dir=__tests__  
(no matches) # perf.ts - 93 lines - imported nowhere
```

IMPACT

The module exists specifically to measure long tasks and dropped frames, which is the evidence for the project's headline frontend claim that a 50 msg/sec feed renders smoothly. Unwired, it measures nothing, and the claim rests on the store benchmark alone — which measures the store, not the render path. By the project's own stated standard this is an adjective without a number.

REMEDIATION

Either wire it to a development overlay behind a query parameter and publish the observed frame statistics alongside the existing benchmarks, or delete it. Both are defensible; leaving 93 lines of unreferenced instrumentation in the tree is not.

M-3 No React error boundary; one throw blanks the entire application

MEDIUM

IMPACT

A single exception inside `PriceChart`, `ZScorePane` or the toast stack unmounts the whole tree to a white page with no explanation. This sits oddly against the backend's carefully considered failure handling and against `StatusBanner`, whose entire design rationale is that a dashboard which fails silently is worse than one showing nothing. The frontend has no equivalent of that principle applied to itself.

REMEDIATION

One boundary around the chart region and one around the toast stack, each rendering a small honest failure state that keeps the rest of the terminal alive. Roughly 40 lines.

M-4 Load suite excluded from CI; benchmark assertions never regression-checked

MEDIUM

EVIDENCE

```
pyproject.toml  
  addopts = '-q --strict-markers -m "not load"'  
  
ci.yml python job  
  uv run pytest tests -q --timeout=90 --cov ... # load never selected
```

IMPACT

The four assertions backing `docs/benchmarks.md` — the O(1) ratio, the conflation memory bound, the 500-client fan-out latency, the replay throughput — are never run automatically. The O(1) claim in particular is the sort of property that degrades quietly: a change introducing an O(n) scan would pass every functional test and only manifest after hours of uptime, which is precisely the failure mode the benchmark exists to catch.

REMEDIATION

Add a fourth CI job running `pytest tests/load -m load`. It need not gate merges — a nightly schedule or a non-blocking job is enough — but the assertions should execute somewhere other than on demand.

M-5 Frontend has no coverage measurement at all

MEDIUM

IMPACT

`vitest.config.ts` declares no `coverage` block, so the 115 frontend tests have no measured line or branch coverage. Given that H-2 found a large untested surface on the backend that the reported number concealed, the frontend's coverage is currently an unknown of the same shape — plausibly good, but unmeasured.

REMEDIATION

Add `@vitest/coverage-v8` and a coverage block; report it next to the backend number. Ten minutes, and it either confirms the suite is as thorough as it appears or reveals a second H-2.

M-6 No contribution scaffolding — the thing GSoC organisations assess

MEDIUM

EVIDENCE

<code>CONTRIBUTING.md</code>	MISSING
<code>CODE_OF_CONDUCT.md</code>	MISSING
<code>SECURITY.md</code>	MISSING
<code>.github/ISSUE_TEMPLATE/</code>	MISSING
<code>.github/PULL_REQUEST_TEMPLATE.md</code>	MISSING

IMPACT

Both target programmes evaluate whether a project is one that others could join. The README is strong on architecture and weak on the question a prospective contributor actually asks first, which is how to get the stack running and what a good pull request looks like here. The information largely exists — the Makefile, the compose file and the ADRs contain most of it — but it is not assembled into the file people look for.

REMEDIATION

A short `CONTRIBUTING.md` covering local setup, the test tiers and how to run each, the ADR convention, and the expectation that behavioural claims come with tests. Contributor Covenant for the code of conduct. A brief `SECURITY.md`. Half a day at most, and it changes how the repository reads to exactly the audience it is for.

M-7 WebSocket authentication is off by default and unthrottled when on

MEDIUM

EVIDENCE

```
fx_api/config.py
    require_ws_ticket: bool = Field(default=False, ...)

fx_api/routers/stream.py
    token = await tickets.issue("anonymous")    # POST /ws/ticket, no auth, no limit
```

IMPACT

With the default configuration the stream is open to anyone who can reach the port. With the flag enabled, `/ws/ticket` still issues a ticket to any caller with no authentication and no rate limit, so the ticket mechanism raises the effort of connecting without materially restricting who may. The design is sound — the code comment correctly explains why a ticket exists rather than a header on the WebSocket — and the `subject` field is clearly the intended extension point. The gap is that the demo posture is the default posture, and nothing in the repository states the deployment expectation.

REMEDIATION

Default `require_ws_ticket` to `True` and make the anonymous subject an explicit opt-in for local development. Add a modest rate limit on the ticket endpoint. Document the posture in `SECURITY.md` (M-6) so the intent is on the record rather than in a comment.

M-8 No rate limiting on the compute-heavy estimator endpoint

MEDIUM

IMPACT

`/api/volatility/{symbol}/compare` reads up to 1,440 bars from Redis and computes five estimators per request, with no throttling. The frontend polls it every 15 seconds for every mounted panel, so ordinary use already generates steady load, and a trivial loop generates arbitrary load. Redis is the shared dependency of the entire system, which makes this the cheapest available way to degrade everything at once.

REMEDIATION

A per-IP limit via `slowapi` or equivalent, plus a short-TTL cache on the computed result — the estimators only change when a bar seals, so a 30-second cache is both correct and sufficient to remove the amplification.

Low

LOW-SEVERITY FINDINGS

ID	FINDING	DETAIL AND REMEDIATION
L-1	Supabase migration path never executed	<code>002_supabase_partman.sql</code> exists because ADR-0007 documents that Supabase ships only Apache-2 TimescaleDB without continuous aggregates. The fallback has never been run against a Supabase instance, so the documented alternative is untested. Either execute it once against a free-tier project and record the result, or mark it explicitly as unverified in the ADR.
L-2	No <code>HEALTHCHECK</code> on the Python image	<code>Dockerfile.web</code> defines one; <code>Dockerfile.python</code> does not. Compose therefore cannot distinguish a wedged ingestor from a healthy one, and dependent services will start against a process that is running but not working. Both the ingestor and worker already expose Prometheus endpoints on 9100/9101 that would serve as a probe target.

ID	FINDING	DETAIL AND REMEDIATION
L-3	No skip link	Keyboard users traverse the header and the full tablist before reaching content on every page load. Compounded by H-4, which removes the main landmark that would otherwise offer an alternative route. A single anchor and eight lines of CSS.
L-4	No dependency vulnerability scanning	286 transitive npm packages and 15 direct Python dependencies, with no <code>pip-audit</code> , <code>npm audit</code> or Dependabot configuration. Both lockfiles are committed, which is the prerequisite; the scanning step is missing. Two workflow lines and one Dependabot configuration file.
L-5	Vestigial directories and stale artefacts	<code>apps/web/src/features/</code> is empty. <code>_to_delete/</code> still holds two delivery tarballs (321 KB); it is now excluded by <code>.gitignore</code> but remains on disk, and the desktop bridge cannot remove it. Delete both by hand.

6 Verification results

Everything in this section was executed during the audit on the environment stated on the cover. Nothing is quoted from previous reporting.

6.1 Quality gates

GATE	COMMAND	RESULT
Backend tests	<code>pytest tests</code>	162 passed PASS
Load tests	<code>pytest tests/load -m load</code>	4 passed PASS
Frontend tests	<code>vitest run</code>	115 passed, 9 files PASS
Type check (Python)	<code>mypy packages apps</code>	48 files, no issues PASS
Type check (TS)	<code>tsc --noEmit</code>	Clean PASS
Lint	<code>ruff check .</code>	Clean PASS
Format	<code>ruff format --check .</code>	79 files PASS
Production build	<code>vite build</code>	333.7 kB JS / 107.2 kB gzip PASS

6.2 Coverage by module — measured across all four packages

Live Redis and PostgreSQL; Timescale-specific DDL stripped per §1.3. Bars are proportional to coverage; red marks modules never executed.

MODULE	STMTS	COVER
<code>ingestor/pipeline.py</code>	163	98 %
<code>ingestor/supervisor.py</code>	94	99 %
<code>api/ws/conflator.py</code>	59	98 %
<code>core/alerts/hysteresis.py</code>	161	98 %
<code>core/alerts/detector.py</code>	99	96 %
<code>core/volatility/range_estimators.py</code>	55	98 %

MODULE	STMTS	COVER
core/calendar/fx_sessions.py	59	95 %
core/alerts/seasonality.py	67	93 %
core/alerts/baseline.py	102	89 %
ingestor/leader.py	67	88 %
core/models.py	95	82 %
core/volatility/ewma.py	47	81 %
worker/db.py	83	69 %
worker/alerts.py	141	68 %
worker/persister.py	170	65 %
ingestor/providers/tiingo.py	94	61 %
core/volatility/buckets.py	69	55 %
api/ws/manager.py	107	50 %
ingestor/providers/replay.py	70	40 %
ingestor/main.py	129	0 %
api/routers/stream.py	81	0 %
worker/main.py	50	0 %
api/main.py	42	0 %
api/routers/health.py	42	0 %
api/routers/market.py	41	0 %
api/ws/protocol.py	29	0 %
api/ws/tickets.py	17	0 %
Total	2,478	64 %

6.3 Performance — re-measured

BACKEND, FROM TESTS/LOAD

PROPERTY	MEASURED	READING
Volatility hot path, first 100k ticks	801 ns/tick	Ratio 0.98× confirms O(1); a naive window would climb steadily and degrade only after hours of uptime
Volatility hot path, after 900k ticks	788 ns/tick	
Conflation, 200k updates to a slow client	5 entries held	Memory bounded by subscribed symbols, not by message rate
Fan-out to 500 clients, p99	533 μs	No client blocks the loop
Replay provider throughput	840 / 2,000 per s	Shortfall is <code>asyncio.sleep()</code> granularity, and the document says so

FRONTEND STORE, MEASURED DURING THIS AUDIT

SCENARIO	MEAN	P99	FRAME BUDGET
50 ticks across 5 pairs, 1 subscriber each	2.4 µs	5.2 µs	0.014 %
500-tick burst conflated into one frame	9.7 µs	28.9 µs	0.058 %
50 ticks with 100 subscribers	5.3 µs	47.9 µs	0.032 %

A tenfold increase in message rate costs roughly four times the work, not ten — conflation collapses a burst to one entry per symbol regardless of its size. That sub-linearity is the property worth quoting, and it is the one that makes the architecture's claim credible rather than the absolute figures, which are small enough to be uninteresting on their own.

7 Recommended sequence

Ordered by dependency and by return on effort, not by severity alone. The first block is short and unblocks everything else.

ORDER	FINDINGS	ACTION	EFFORT
1	C-1, C-2	Initialise git, add a licence, push, watch CI run. Nothing else in this register can be verified until the pipeline executes for the first time, and the two Critical findings both close here.	1 hour
2	H-3	Make service-dependent fixtures fail rather than skip when <code>FX_REQUIRE_SERVICES=1</code> . Do this <i>before</i> trusting the first green build, or the first build may be green for the wrong reason.	30 min
3	H-4, L-3	Fix the landmark structure, add the skip link, and add one shell-level axe test so composition defects cannot recur.	1 hour
4	H-2	WebSocket and REST endpoint tests via <code>TestClient</code> . The single largest quality gain available, and it closes the biggest gap a reviewer would find.	Half a day
5	H-1, M-5	Widen coverage scope to all packages, add frontend coverage, set a <code>fail_under</code> ratchet at the honest number.	1 hour
6	M-6, M-7, M-8	Contribution scaffolding, security posture documented and defaults tightened, rate limiting on the estimator endpoint.	1 day
7	M-1, M-2, M-3	Playwright smoke suite, wire or delete <code>PerfMonitor</code> , add error boundaries.	1 day
8	M-4, L-1, L-2, L-4, L-5	Load job in CI, Supabase path verified or marked unverified, container health check, dependency scanning, tidy-up.	Half a day

On the shape of this list. Roughly three days of work closes every finding in this register, and the first ninety minutes of it closes both Critical findings and the two that most affect how the project is perceived. That ratio is itself a finding: this is a project whose remaining risk is concentrated almost entirely in things that were never started, rather than in things that were done badly. That is a substantially better position to audit than the reverse.

8 Appendix — reproducing this audit

Every measurement above can be regenerated. Services are assumed on default ports.

```

# Quality gates
uv run pytest tests -q
uv run pytest tests/load -m load -q
uv run mypy packages apps
uv run ruff check . && uv run ruff format --check .
cd apps/web && npx tsc --noEmit && npx vitest run && npm run build

# Honest coverage – all four packages, not just the core
uv run pytest tests -q \
  --cov=packages/core/fx_core \
  --cov=apps/api/fx_api \
  --cov=apps/ingestor/fx_ingestor \
  --cov=apps/worker/fx_worker \
  --cov-report=term

# Frontend store benchmark
cd apps/web && npx vitest bench --run

# Version control status (finding C-1)
git rev-parse --is-inside-work-tree

# Licence presence (finding C-2)
ls LICENSE LICENSE.md 2>/dev/null || echo MISSING

```

Note on the accessibility finding

H-4 was reproduced by rendering a faithful copy of `App.tsx`'s landmark structure — header, tablist, `<main role="tabpanel">`, footer — through `jest-axe` and reading the violation list, then re-running the corrected structure to confirm it reports none. That test does not currently exist in the repository, which is precisely the gap the finding describes; adding it is part of the remediation.